31263 / 32004 Introduction to Game Development

Assessment 3 and 4
Recreate a Classic Game

Overview:

Assessment 3 and 4 are two parts of a linked assessment. In these assessments you will recreate a specified classic Atari-era game by developing nearly all the assets yourself. The game will look and feel different to the original but will have the same core game design. You will then innovate on the game to push your self-study skills and explore new game development topics on your own. These are **individual assessments**, and it is expected that all the work done here is your own.

Primary Design and Development Constraints:

1. You must use the Unity version that is used in the lab room and specified on Canvas.
2. The game must be made in 2D, using sprites.
3. Your final game (in Assessment 4) will have at least three scenes – one that is for the main menu, one that is a recreation of a level from the original game, and the last that shows off your design innovation. You may have more scenes in your project structure if you wish.
4. All visual assets must be entirely your own creation. These must be ***significantly different*** in appearance to the original game, and they must be your own designs so as not to infringe on copyright or trademark. i.e. do not draw existing characters from games, anime, movies, memes, etc.
5. You must have sprite animations in your game. All animation clips must be visibly animated. You may also create your own particle system effects with 2D sprites.
6. Audio must be included for all major interactions in the game, but the audio clips do NOT need to be your own creation. They must be different to the original game, but you may source these from royalty-free audio websites.
7. All scripts in the game must be your own creation. You may use learning resources such as the Unity manual, forums, Unity Answers, and video tutorials, but you must not use any downloaded scripts from the Unity Asset Store, GitHub, or similar. Sophisticated plagiarism checking software will be used to ensure that your code has not been substantially copied from any single source, whether that is other students or online resources.
    a. This also extends to GenAI approaches like ChatGPT, Claude, Copilot, etc. You can query them or ask for advice, but do not copy the code they produce wholesale.
    b. If GenAI is used for this assessment, you must make it clear what prompts were used and the outputs that were utilized.
8. You may NOT use the Unity Rigidbody physics functionality or the standard Unity CharacterController component (or similar). Therefore, all motion must be coded by you as frame-rate independent continuous movement (see Week 5 lecture). If you have a Rigidbody

or Rigidbody2D component on a gameobject for the purposes of collision detection, it must be set to "Is Kinematic" at all times (or Body Type set to Kinematic if you are using a Rigidbody2D).

9. You MAY use the Tilemap functionality in Unity, though it is not required. You MAY NOT use the extra Tilemap Animated Tile functionality as it will cause issues in Assessment 4. For Assessment 3 Band 75% D, you should place any animated tiles manually, regardless of whether you used Tilemaps or not.
10. Assessments 4 stacks on Assessment 3 – anything you miss in Assessment 3 will likely affect your Assessment 4 grade as well.

Assessment 4 – 35%

Complete PacStudent and Design Iteration

Due Dates:

Due via Canvas before **11:59pm Friday Week 12**

Deliverables:

Unity Project Files:
studentNumber_A4_Project.zip

This is your entire Unity project folder (including your .git folder and .gitignore file), zipped up (as a .zip, no other format), with the specified naming convention. Before creating the zip, delete the "Library" folder.

Student Submission Declaration:
studentNumber_A4_Declaration.txt

A separate submission portal will be open on Canvas to submit a declaration. On this page, you will find a template file to complete and then submit. This template will be similar to the COMPLETE_BEFORE_SUBMISSION.txt files you have completed for each lab activity.

Extensions:

Extensions will only be granted in the case of extenuating circumstances. You must provide evidence of this, such as medical certificates, official letters, photos or video of incident, etc. That you have found the assessment hard or mismanaged your time is not an extenuating circumstance.

Extension requests should be sent to the Subject Coordinator (dale.grant@uts.edu.au). A maximum of 1 week extension can be granted (though the granted time will be proportional to the circumstances. For longer extensions, you must submit an official request for Special Consideration (https://www.uts.edu.au/current-students/managing-your-course/classes-and-assessment/special-circumstances/special-consideration).

Plagiarism Detection Software:

Sophisticated code plagiarism software will be used to check all code submissions in Assessment 3 and 4 with those of other students in the subject, submissions from previous years, and online sources. While learning from online tutorials and communities is strongly encouraged, there is a difference between learning code segments and re-implementing the included concepts or specific Unity API calls, versus plagiarising large chunks of code. A similarity score higher than 25% between a submitted project and any single other source will be further investigated by the Subject Coordinator and, where necessary, raised with the university as Student Misconduct. Additionally, visual and audio assets will be manually checked for originality by marking staff.

You should not be using any assets, packages, or plugins other than those that come with the Unity Installation or that you have created yourself (except the audio assets). This includes any add-on packages found from websites like the Unity Asset Store.

Task and Grade Overview:

In this assessment you are going to be extending upon what you started in Assessment 3 and making the complete PacStudent game. The table below is structured like the lab activities. You should start from the top and work your way downwards. These tasks are stacked in terms of difficulty as well as completing requisite functionality before moving on to later functionality that utilizes this.

You must complete each grade band before moving onto the next. If there are major issues with an earlier section, we will not mark the later ones. E.g. If you have made some ghost behaviors but your PacStudent movement is barely working, you will only be marked up to and including the PacStudent movement level. Likewise, if you skip all of the earlier steps and go straight to the 100% HD Design Iteration, then you will only be marked to the 30% Z level. This is to encourage students to do good, consistent work throughout each section before moving onto the next and stopping students from thinking they can get easier grades by skipping steps.

This assessment is meant to be challenging.
We do not expect everyone to aim for the 100% HD (or even the 85% HD or 70% C). We follow UTS policy in that a D should be good work, HD should be exceptional work, and a 100% HD is outstanding or near perfect work. Thus, the 100% HD level in this assessment is there to encourage those that want to push themselves. Making games is hard, for everyone. It is a “rockstar” industry

that attracts the best-of-the-best software engineers and artist or those with an unwavering passion for games. To succeed in this industry, you always need to be pushing yourself to be better and continue to learn new skills and solve previously unseen problems throughout your life. This subject is no different. We want our students to graduate this degree with as many skills as possible and prepare them for the competitive industry. No matter which degree you are in, push yourself to the highest level you can and be proud of whatever personal achievement that you make for yourself.

Penalties:

The below penalties have been added to discourage certain trends that we have noticed in student work in the past. These penalties can be applied to any level that you get up to, reducing your overall grade:

- Red Compilation or Runtime Errors: If, when opening your project or when playing the game, there are any red errors in the Console Window, you will lose 2 points for each distinct error. This includes compilation errors, IndexOutOfRangeException, NullReferenceException, and others. This does not include Unity Engine errors, which usually come when you first open a project and will disappear if you hit the "Clear" button on the Console Window. We will not seek to break your game, so these errors should not appear in typical gameplay. You may think your game is fine with these errors, but it is not, it is very unsafe to leave them and seeing them in submitted work is a sign of an unprofessional developer. As soon as you see an error (or even a yellow warning) during your development, stop everything you are doing and fix it.
- The entire game is on a UI Canvas: There are elements of this assessment that require you to use a UI Canvas. However, only those UI/HUD elements should be on the Canvas. You will lose 4 points if the level, characters, pellets, and bonus cherries are on a UI Canvas. If you completed Assessment 3 entirely on a UI Canvas, you will need to remove this and re-create your level in World Space as a collection of GameObjects.
- Resources Folder and Resources.Load(): You will lose 4 points if the Resources folder and/or Resources.Load() is used for anything other than special circumstances in the 100% HD Design Iteration. Some of you have picked up the bad habit of using the Resources folder. This folder is meant for assets that are created at runtime, such as procedurally generated content or player made content, as well as some other rare cases. It should not be used to store sprites, audio files, prefabs, or any other typical game asset. This is because anything in the Resources folder:
    - Must be manually unloaded from memory (e.g. Destroy() and Scene.Load() will not automatically free up the memory)
    - Is kept in its raw form and not compressed with the other game files, making your game larger (a big issue if you are distributing to a mobile store) and making it easier for your assets to be pirated.
    - Causes runtime importing of the assets. When you add an asset to Unity, it is imported to the engine with the settings that can be seen in the Inspector Window. Doing this at runtime will just reduce the framerate of your game.

Overview Videos:

See the video overviews on the Assessment 4 page of Canvas for each grade band to get a better understanding of each. Wherever any discrepancies exist between the videos and this assessment specification document, follow the instructions within this document.

Max grade possible	Requirements
3.5 (10%)	**Git Repository** ● You will again be required to use your Git repository in this assessment. You can continue to use your repository from Assessment 3. ● The marks for the repository will be determined in relation to the band you reach. That is, the number of branches and commits you have should be proportional to the grade band that you reach. ● You should have a main branch, a dev branch, and at least one feature branch per grade band that you complete. ○ The dev branch should stem from the main branch. ○ Each feature branch should stem from dev, unless you have multiple feature branches for a single grade band, in which case subsequent feature branches may stem from the first feature branch in that band. ○ You should make multiple commits per feature branch and merge the feature branches back into dev after the band is completed. ● Dev should be merged back into the main branch before submitting your assessment. ○ Make sure the main branch is checked-out before submitting so that your final files are visible to markers. ○ Your marker should not have to go searching for your completed work! ● The repository must be connected to a remote online server (e.g. GitHub or BitBucket). You do not need to share this or provide account details, it will be evident from the Git folder whether you have a remote server reference. Additional resources and suggestions: ● For more on this style of branching structure, see the Git Workflow branching strategy. You may have more branches if you wish. ● It is highly recommended that you work on one branch/feature/grade band at a time. E.g. checkout dev -> branch Feature-Assess3Catchup -> work on fixing Assess3 work while committing frequently -> finish and merge Feature-Assess3Catchup back into dev -> branch Feature-StartScreen -> repeat. ○ This will help you to avoid merge conflicts. Git helps you recover from poor planning (e.g. resolving merge conflicts), but it is much harder and time-consuming than just planning your project flow ahead of time. ● If something goes wrong - https://dangitgit.com/

	○ Git is there to protect from catastrophic failure, as long as you commit and push often. ○ If you mess up a merge, reset back to a previous commit. ○ If your PC malfunctions, clone the repository from your remote (e.g. GitHub) to another computer. ○ With this in mind, **no extension requests will be granted for reasons of having PC, Unity, project, or Git issues/crashes.**
7 (20%, Z)	**Catch-up from Assessment 3** ● If you were unable to finish Assessment 3, complete as much as possible in any way possible, up to and including the 75% D band (Manual Level Layout). ● The Ghosts should be placed in their center starting area, where they will start the game. ● PacStudent should be placed in the top-left hand corner, where the player will start the game. ● The power pellet flashing animation should be playing. ● Remove the movement component for PacStudent from Assessment 3. ● Remove the ghost animation cycles (but don't delete the animator from the Project View, you may need it later). ● Remove any excess sprites that aren't part of the core level (PacStudent, Ghosts, Walls, Pellets, Power Pellets, etc sprite examples).
10.5 (30%, Z)	**Start Screen UI Appearance and Functionality** ● Create a new scene called "StartScene". This will be the first scene of the game and it will be your title screen. ● Create a title screen that has at least the following elements (though you can add more if you wish. As always, this should be in your own distinct style, not a replication of the original Pacman title screen): ○ The title "PacStudent" wording. ○ A creative subtitle for your game. ○ Your "PacStudent" . ○ Your "Ghost" sprites. ○ An animated border: ▪ For example, moving dots around the title or anything else you can think of. The border needs to be clearly animated in some manner. ▪ The border should be a canvas element and scale with the aspect ratio as well! ○ "Level 1" and "Level 2" buttons. ▪ When Level 1 is pressed, load up your Assessment 3 scene (that you will build upon in this assessment).

	▪ When the Level 2 button is pressed, load the Design Iteration scene (see the 100% HD below). If you do not get to the 100% stage, this button does not need to do anything. ▪ Buttons should react to being highlighted and clicked, visually and/or aurally. ○ Previous high score and time for each level. ▪ These can just be 0 and 00:00:00 respectively for now. You will need to adjust them later when doing high score saving and loading. ● This scene should be playing the "Start Scene" background music from Assessment 3, on a loop. ● Aspect Ratio: The Start Screen menu should scale to fit both 16:9 and 4:3 aspect ratios in landscape mode. Make sure to test this before submitting!
14 (40%, Z)	**In-game UI Appearance** ● Continue to create your PacStudent re-creation by building upon the scene you were working on in Assessment 3. ● Add a Screen Space Canvas called "HUD" and add the following elements to the canvas. Aside from the Exit button, nothing else needs to be functional yet, just appear on screen: ○ Lives: A counter using the "Life Indicator" sprite from Assessment 3 ▪ The player should start the game with 3 lives and should be represented using the aforementioned sprite, not a number. ○ Score: A text field that shows the current numeric score and starts at 0, with padding of zeros to six digits. E.g. 1234 is shown as "001234". ○ Game Timer: A text field that shows a numeric timer in the form of mm:ss:msms (e.g. 01:32:25 is 1 minute, 32 seconds, and 250 milliseconds). This should display 00:00:00 at the start of the game. ○ Ghost Scared Timer: A text field that shows seconds as an integer that will appear and count down when the ghosts are in the scared state. The timer value should be invisible at the start of the game or when the value is zero. ○ Exit Button: When pressed, the game should go back to the Start Scene from the previous section. ○ Level Name: The name of the current Level. This should be something like "Level 1" (or "Level 2" for the HD Innovation Level). ● Aspect Ratio: The in-game HUD should scale to fit both 16:9 and 4:3 aspect ratios in landscape mode. ● Add four World Space Canvases called "Ghost1Canvas" through to "Ghost4Canvas": ○ Each ghost should have one of these world space canvas as a child object.

	○ Each canvas should have a single text box that has a number from 1 to 4 in it such that your ghosts are labeled 1, 2, 3, and 4 in the game view and should be ***clearly*** readable above the ghost.
19 (55%, P)	**PacStudent and Cherry Movement** ● Create a script called "PacStudentController.cs". In this script, implement PacStudent's movement in the following way: ○ Use a linear lerping/tweening approach to move from one grid position to another. This grid should align with the LevelGenerator.cs levelMap. I.e. PacStudent should be lerping from one pellet position (or empty space) to another at a fixed speed and be frame-rate independent. ○ You may NOT use external tween libraries (such as DoTween). If these libraries are found in your project structure, you will be severely penalized. ○ In Update(), gather player input for moving with the W, A, S, and D key to move PacStudent up, left, down, and right respectively. ▪ Store the last key that the player pressed in a member variable called "lastInput". ▪ Do not override/clear this variable's value unless the player provides a new input! ○ In Update(), if PacStudent is not lerping (i.e. was not previously moving or has just reached a grid position), then… ▪ Check lastInput and try to move in that direction to the adjacent grid position. ▪ If the adjacent grid position from lastInput is walkable, then store lastInput in a member variable called "currentInput" and lerp to the adjacent grid position. ▪ If the adjacent grid position from lastInput is not a walkable area (e.g. a wall), then... ● Check currentInput (see below) and try to move in that direction to the adjacent grid position. ● If the adjacent grid position from currentInput is walkable, then lerp to the adjacent grid position. ● If the adjacent grid position from currentInput is not a walkable area (e.g. a wall), then do nothing (PacStudent stops moving). ○ The Ghost Wall tile acts as a wall for PacStudent and ghosts outside the ghost start zone, but ghosts can pass through from the inside. ▪ When the game starts, PacStudent should be facing to the right, awaiting a new player input. ○ With this setup, PacStudent's movement should feel like the reference example except with a slight delay in reaction time if the player

	provides input while PacStudent is lerping! https://www.google.com/search?q=play+pacman+doodle ○ The move speed of PacStudent should be around 5 tiles per second for a fast game experience! ○ The Week 9 Workshop will go over the specifics of the movement and a visualization of the above "algorithm". ▪ No, you may not do PacStudent's movement in any other way. ▪ No, you may not use Rigidbody physics to handle movements. ▪ Yes, we know it isn't the common way of doing things, that is the point. It is unique and we are checking to see if you can program unique systems without following the thousands of unimaginative Unity tutorials out there that lead to the same uninspired feeling in games. ▪ I recommend adapting the Tweener/Tween system we implemented in the labs for this movement system. ● Make sure that PacStudent's movement animations and audio play when they are lerping, and stop when they are not moving! ○ Remember that there are two movement audio clips – one for when PacStudent is eating a pellet (or about to eat one) and one for when they are moving but not about to eat a pellet in the next grid position. ● Create a custom dust particle effect and play it around PacStudent when they are moving so that it looks like they are running through dirt (or a similar surface). ● Create a script called "CherryController.cs" to implement the spawning and movement of the bonus cherry sprite created in the previous assessment. ○ The bonus cherry should spawn 5 seconds after the previous cherry has been destroyed (and after 5 seconds of the scene starting). ○ It should be instantiated at a random location just outside of the level map but only be visible when inside the bounds of the level (Hint: look into sprite masks). The starting location should be different every time. ○ It should move in a straight line, via linear lerping, from one side of the screen to the other, passing through the center point of the level and ignoring collisions with walls, ghosts, and pellets. The bonus cherry should be drawn over all other sprites in the level. ○ If the cherry reaches the other side of the level, outside of camera view, destroy it. ○ See below for what to do if PacStudent collides with the cherry.
24.5 (70%, D)	**Collisions, In-game UI Functionality and Saving High Scores** ● Register collisions between PacStudent and the following elements with the following effects: ○ As per to original rules in the Assessment 3 specifications, you may only use Rigidbody components to help with registering collisions (e.g.

to allow OnTriggerEnter() method calls to between more objects, refer to the Unity collision/trigger matrix on https://docs.unity3d.com/Manual/CollidersOverview.html).

- You may NOT use the Rigidbody to handle collision responses (e.g. to put force onto Pacman).
- Therefore, any Rigidbody components in your game must be set to IsKinematic = true in the Inspector Window (or Body Type set to Kinematic if you are using a Rigidbody2D).

- Walls:
  - Prevent movement in that direction and (if need be) make sure PacStudent is moved back to the previous lerp position before the collision happened.
  - Create a small particle effect and play it from the point of collision of the first frame of the collision so it looks like PacStudent has bumped into the wall.
  - Play the wall collision sound effect on the first frame that PacStudent collides with the wall.
- Teleporters:
  - When PacStudent gets to the end of one of the empty tunnels on the left and right side of the map, move them to the tunnel on the other side.
  - After teleporting, PacStudent's movement should continue from that point, moving inwards towards the rest of the level.
- Pellets: Destroy the pellet and add 10 points to the player's score and update the UI element for this.
- Bonus Cherry:
  - Destroy the cherry and add 100 points to the player's score.
  - Play the sound effect audio for eating a cherry when the cherry is eaten.
- Power pills:
  - Add 50 points to the player's score.
  - Change the Ghost animator state to "Scared".
  - Change the background music to match this state.
  - Start a timer for 10 seconds. Make the Ghost Timer UI element visible and set it to this timer.
  - With 3 seconds left to go on this timer, change the Ghosts to the Recovering state.
  - After 10 seconds have passed, set the Ghosts back to their Normal states and hide the Ghost Timer UI element. A Dead ghost should not be changed to the Normal state.
- Ghosts – Normal State:
  - PacStudent dies and loses a life. Update the UI element for this.

- Play a particle effect around PacStudent the spot of PacStudent's death and the appropriate Pacstudent animation. Ghosts should not move during the Pacstudent death sequence.
- Play the pacstudent death sound effect that should play in time with the PacStudent death animation.
- Respawn PacStudent by moving them to the top-left hand corner, where they started the game, and wait for player input. Ghosts are reset to their initial positions and to the Normal State.

  - Ghosts – Scared and Recovering States:
    - The ghost dies and enters their Dead animator state.
    - Change the background music to match this state.
    - Play the GhostEat sound effect when successfully eating a ghost.
    - Add 300 points to the player's score.
    - Start a 3 second timer (this does not need to be visible). Once this 3 seconds has passed, transition the ghost back to the Normal state if the ghost scared timer has finished; otherwise, the ghost enters either the Scared or Recovering states depending on the remaining time of the scared ghost timer.

- Round Start:
  - At the start of the round, on the HUD canvas, show a big countdown of "3", "2", "1", "GO!" (each displayed 1 second apart). There should also be a semi-opaque image that obstructs all other screen elements and is hidden by default and enabled while the countdown text is visible.
  - While the countdown is progressing, play the game intro background audio, which should stop when the countdown text disappears as described below.
  - During this time, the Game Timer (below) should remain at 0 and the player and the ghosts should not be able to move or collide with anything (see previous and later sections).
  - After "GO!" has been shown for 1 second:
    - Hide this UI element and the blocking image and start the game.
    - Enable player control and ghost movement (if you complete the 90% section below).
    - Start the background music for when ghosts are in their Normal state, which should loop if it finishes.
- Game Timer:
  - Make the timer functional. It should start at 00:00:00 and count upwards after "GO!" disappears from the screen.
- Start Scene High Score and Time:

	○ In the Start Scene, when the game starts, use PlayerPrefs to load the previous high score and associated time and update the UI element for this (see Game Over below for saving the score) ● Game Over: ○ When either every pellet is eaten, or PacStudent has no lives left, show "Game Over" text on the HUD canvas with a blocking image as in the Countdown timer above. ○ Stop all player and ghost movement and pause the Game Timer and prevent usage of any UI buttons. ○ Use PlayerPrefs to save the player's current score and current time if the current score is greater than the previous high score or if the score is the same but the time is lower than the previous best time. ○ The "Game Over" text should remain visible for 3 seconds and then return the player to the Start Scene. If the high score has changed, make sure to update it on this scene.
29.5 (85%, HD)	**Ghost Movement and Artificial Intelligence** ● Place all four ghosts in the center area of the map, evenly spaced in a horizontal line. ● Create a new script called "GhostController.cs" to handle ghost movement. ● This should work in the exact same way as the player movement, except instead of waiting for player input, at the end of each lerp each ghost should make a decision about where to move next based on the following: ○ No Unity Pathfinding: You are NOT allowed to use the Unity Pathfinding tools. ○ No stopping: In all cases below, the ghosts never stop moving. ○ No backstep: In all cases below, ghosts cannot move back to a grid position that they just came from (i.e. they cannot reverse direction) unless there is no other choice. ○ No walking through walls: The ghost should only move to "valid" positions, such as those that PacStudent can walk on (e.g. the corridors). ○ Ghost cannot teleport: In all cases below, ghosts cannot use the teleporters. They treat them as walls and force back-tracking. ○ Dead state ghosts: Move in a straight line, at a constant, frame-rate independent speed, towards the ghost spawn area. ▪ Dead ghosts move through all walls and ignore all collisions with PacStudent. ○ Ghosts respawning: If a ghost is in the spawn area and they are in a… ▪ Dead state: ● Reset the ghost state to Normal, Scared, or Recovering to match the global ghost state once they have returned to their starting position.

	● If no other ghosts are in the Dead state, change the background music to match the current global ghost state. ▪ Normal/Scared/Recovering state: Move directly to leave the spawn area out (one) of the gaps in the spawn area walls. Ghosts 1 & 3 are to move out of the top gap, while ghosts 2 & 4 leave through the bottom gap. ▪ After exiting the spawn area, ghosts cannot re-enter unless they are in their Dead state. ▪ Disable the Dead Ghost Timer from the previous section, ghosts should only become undead upon reaching the spawn area or when the game resets upon PacStudent death. ○ Normal state ghosts: Match the below ghost behaviors with the number above each ghost's head (see in-game HUD – world space section above). ▪ Ghost 1: Move in a random valid direction such that PacStudent will be either further than or equal distance to the ghost (straight-line distance) when compared to the ghost's current position. ▪ Ghost 2: Move in a random valid direction such that PacStudent will be either closer than or equal distance to the ghost (straight-line distance) when compared to the ghost's current position. ▪ Ghost 3: Move in a randomly selected, valid (not back-tracked) direction. ▪ Ghost 4: Move clockwise around the map, following the outside wall. ○ Scared and Recovering state ghosts: All ghosts use Ghost 1 behavior. ○ Movement Speeds: ▪ Normal: 90% PacStudent. ▪ Scared: 50% Normal Ghost Movement Speed. ▪ Recovery: Same as Scared Movement Speed. ▪ Dead: Same as Scared Movement Speed.
35 (100%, HD)	**Design Innovation** This is an opportunity for students to spread their wings and get creative. Once you have completed everything above, create a second scene called "InnovationScene" and link it with the "Level 2" button on the main menu. In this scene, design some meaningful gameplay innovation on top of the existing PacStudent experience.

	This may include (but is not limited to) ONE of the following: ● A weapon / abilities / pickup system that enables extra controls for PacStudent. ● A small assortment of slightly altered game modes that change the player's objective or rules of the game (see the game "Move or Die" as an example). ● A single significantly different game mode that fundamentally changes the PacStudent experience. ● Enhanced visuals, particle effects, audio, and camera movement for better immersion. ● Procedural generation of the level or other content in the game to increase replayability. ● Enhanced ghost AI that behaves more strategically. ● A full 3D remake of the game with either a 1st or 3rd person camera angle. ● Etc. If you have another idea and you are unsure whether it is too small or too large for the scope of this assessment, talk to the lecturer or tutors about it. Fill out the Assessment 4 Submission Declaration text file and submit to the appropriate portal on Canvas. If you complete this section, make sure to fill out in detail the "Design Innovation" section. Explain what your innovation is and how it transforms the player experience from the standard pacman recreation level!